Web Hangman — Development Report

Name: Kris Rimal
Course: CSE 4095
Repository: https://github.com/MaybeSword/web-hangman
Live app: https://hangman-n5f4.onrender.com

1. Development Process

I built the project with an AI coding agent. In August 2026 I first used Cursor to build a fuller version phase by phase: I set up git, deployed to Render, and fixed several Render deployment errors one at a time. That code was later not available on my machine. In November 2026 I rebuilt this repository with Claude Code from the original `SPECIFICATION.md` and `TODO.md`. The agent first revised `TODO.md` into five phases: engine and words, Flask API, browser client, testing, and deployment. Larger features such as the terminal client, bots, evaluation mode, scoring, leaderboard and replay were marked out of scope. For each phase the agent implemented the code, ran the tests and checked the result in a real browser. A TODO item was checked off only after it had been verified.

2. Implementation

Game engine (`game.py`). `HangmanGame` tracks the word, guessed and incorrect letters, the remaining guesses (6 by default) and the status (`playing`/`won`/`lost`). It raises `InvalidGuess`, `DuplicateGuess` or `GameOver` for bad moves. `WordManager` loads the `easy`/`medium`/`hard` word lists from `words/` and picks a random word. The module does not import Flask.

Flask server (`server.py`). `create_app()` is an app factory, so tests can inject their own word manager. Games are kept in an in-memory dictionary keyed by an unguessable `secrets.token_urlsafe(8)` ID. The server also serves the client at `/`.

REST API. `POST /api/games` (201; 400 for a bad difficulty), `GET /api/games/{id}` (200; 404 for an unknown game) and `POST /api/games/{id}/guess` (200; 400 for an invalid letter, 404 for an unknown game, 409 for a duplicate guess or a finished game). Errors return `{"error": ...}`. `to_dict()` adds the `word` field only once the status is `won` or `lost`.

Browser client (`static/`). `app.js` talks to the server only through `fetch()` and never has the hidden word during play. The gallows is drawn in SVG. Letters can be entered with the on-screen keyboard or a physical keyboard, and the layout is responsive down to phone width.

3. Testing

There are 24 `unittest` tests, all passing (`python -m unittest discover -s tests -t . -v`). `test_game.py` (13) covers the engine and the word manager. `test_server.py` (11) covers every endpoint and status code, asserts that the word is never in a response while the game is playing, and runs an end-to-end game over real HTTP. I also played full games by hand: locally under gunicorn (every guess POST returned 200 in the DevTools Network tab, with no console errors and a working layout at 375 px width), and on the deployed site (create returned 201 without the word, guesses returned 200, an unknown ID returned 404).

4. Deployment

The app runs on Render's free tier from the `render.yaml` Blueprint. The build command is `pip install -r requirements.txt` and the start command is `gunicorn --bind 0.0.0.0:$PORT --workers 1 server:app`. Gunicorn imports `server:app` directly, so the `app.run()` debug block never runs in production. Render redeploys automatically on every push to `main`. Live URL: https://hangman-n5f4.onrender.com.

5. Challenges

- **Missing original code.** *Problem:* the Cursor code and word lists were no longer available. *Resolution:* I rebuilt the project from the spec and the revised TODO, and wrote new word lists.
- **In-memory state with several workers.** *Problem:* each gunicorn worker has its own memory, so a guess could reach a worker that did not know the game and return 404 "game not found". *Resolution:* I run with `--workers 1` and documented the limitation in the README.
- **Hidden-word leaks.** *Problem:* the client must not be able to read the answer. *Resolution:* `to_dict()` includes `word` only after the game is won or lost, and tests assert that it is absent while playing.
- **Cold starts.** *Problem:* the Render free tier sleeps when idle, so the first request takes 30–60 s. *Resolution:* I documented this in the README.

6. Reflection

The biggest lesson was that an AI agent works best when I give it a narrow, clear scope. Cutting the TODO down to five phases and telling it "do not write code yet" kept it from building features the assignment did not ask for. I also learned not to trust a checkmark on its own: my old TODO had every box checked, but that code was gone, and this time items were marked done only after the tests passed and the game worked in a real browser. The agent was fast at writing the engine, API, tests and deployment config, but some steps still needed me, such as signing in to Render, deciding to make the repository public, and recording the demo. Overall, I now see AI-assisted development as directing and checking the work rather than just accepting what the agent produces.

Representative Prompts

1. "Revise TODO.md into a phased plan for these areas only: game engine and word management, Flask server and REST API, browser client, testing and debugging, cloud deployment. Mark removed tasks as out of scope. Do not write code yet."
2. "Prepare this project for Render. Add gunicorn to requirements.txt. Make sure the app does not run in debug mode in production. Use this start command: `gunicorn --bind 0.0.0.0:$PORT --workers 1 server:app`. Add a short deploy section to README.md. Run the tests, then commit."
3. "Review the project against this checklist: engine, three endpoints, hidden word never leaked, browser uses fetch only, tests pass, README present, original spec and updated TODO in the repo, no secrets committed. List anything missing."